

Zustandsschätzung im Simulator

Praktikumsversuch 2 — Kalman-Filter für GPS, Odometrie und IMU

Betreuung: Robotik-Labor

Version 0.1 — Wintersemester

Sie schätzen, Sie fahren nicht. Ein Bewerter setzt Ihren Roboter an den Start, fährt ihn durch eine vorgegebene Kommandofahrt und zeichnet dabei die exakte Pose (**truth**) und Ihre Schätzung **kf/pose** auf. Urteilt wird über Genauigkeit: RMSE gegen die Wahrheit, Verbesserungsverhältnis gegenüber dem Rohsensor, Maximalfehler während eines GPS-Funklochs und Konsistenz der von Ihnen angegebenen Standardabweichung (NEES). Dazu bauen Sie einen Kalman-Filter mit Zustandsvektor aus Position und Geschwindigkeit, füttern ihn mit GPS (langsam, verrauscht, zeitweise weg), Odometrie (schnell, driftend) und IMU (kurzfristig sauber, langfristig wertlos) und melden *mit*, wie unsicher Sie sind. Der Simulator ist derselbe aus Versuch 1 — neu sind ein IMU-Rauschmodell, drei Themen und die Frage, ob Ihre Zahlen stimmen.

Inhaltsverzeichnis

1. Versuchsziel und Vorfragen	2
1.1. Ziel	2
1.2. Vorfragen (vor dem Praktikumsstermin rechnen)	2
2. Theorie: präzisieren, messen, abwägen	3
2.1. Zustand, Prädiktion, Update, Kovarianz	3
2.2. Das CV-Modell in Matrizen	4
2.3. Warum die Gyro kurzfristig und die Odometrie langfristig hilft	4
2.4. GPS: Rauschen, Bias, Funkloch, Modellfehler	4
3. Der Simulator	6
3.1. Neue Themen und was sie enthalten	6
3.2. Das IMU-Modell und seine Zahlen	6
3.3. Auswertung in der GUI	7
4. Inbetriebnahme	8
4.1. Installation	8
4.2. Werkstatt: den eigenen Filter fahren	8
4.3. ROS-2-Betrieb mit dem Launch-File	8

4.4. Drei Regeln, die den Lauf retten	9
5. Aufgaben K1 bis K4	10
6. Auswertung und Bericht	11
A. Formelsammlung	12
B. Lösungen der Vorfragen	12
C. Themen- und API-Übersicht	13

1. Versuchsziel und Vorfragen

1.1. Ziel

Nach dem Versuch können Sie

- einen Zustandsvektor (x, y, v_x, v_y) mit einem CV-Modell prädictieren und jede Messung mit ihrer eigenen Unsicherheit einarbeiten — Prädiktion, Update und Kalman-Verstärkung aus P , Q und R , nicht aus dem Bauch,
- die eigene Unsicherheit so angeben, dass sie *stimmt*, und sie mit dem normierten Fehlerquadratmittel (NEES) prüfen,
- Odometrie-*Twist* und Gyro-Drehrate als Bewegungsmodell benutzen, um ein GPS-Funkloch von zehn Sekunden unter 1,8 m Fehler zu überbrücken,
- Drift (korreliert, wachsend) von Messrauschen (weiß, mittelwertfrei) trennen und begründen, warum eine IMU niemals offen integriert wird,
- aus einem Messprotokoll RMSE-Zeitreihen, eine Q/R -Tabelle und eine NEES-Diskussion für den Bericht erzeugen.

Rollenverteilung.

Der Bewerter setzt den Roboter vor *jedem* KF-Auftrag neu an der Spawn-Pose ab und fährt die Kommandofolge aus `config/tasks.json` selbst über `cmd_vel`. Ihr Knoten sendet *keine* Radgeschwindigkeiten und definiert keine `inverse_kinematics` — sonst konkurriert er mit dem Bewerter und der Lauf ist wertlos. Gearbeitet wird allein pro Person: ein Robotername, ein Filterknoten.

1.2. Vorfragen (vor dem Praktikumsstermin rechnen)

Verwenden Sie die Simulatorwerte aus table 2 (`mecanum_lab/types.py: DEFAULT_CONFIG`) und die Prüfprofile aus `config/tasks.json`: Rauschendichte $\sigma_a = 0,002 \text{ m/s}^2/\sqrt{\text{Hz}}$, Accelerometer-Bias $b_a = 0,05 \text{ m/s}^2$, IMU-Rate 100 Hz, Radrauschen $\sigma_{\text{wheel}} = 0,04 \text{ rad/s}$, GPS-Streuung 0,5 m. Lösungen: appendix B.

1. **Was dominiert die IMU?** Eine Rauschendichte von $0,002 \text{ m/s}^2/\sqrt{\text{Hz}}$ bei 100 Hz: welche Standardabweichung hat *eine* Stichprobe, und warum steht dort $\sqrt{\text{rate}/2}$?

Rechnen Sie den Wegfehler aus, den ein fester Bias von $0,05 \text{ m/s}^2$ nach 10 s Doppelintegration liefert, und vergleichen Sie ihn mit dem Beitrag des weißen Rauschens nach derselben Zeit. Was folgt daraus für die Filterpraxis?

2. **Rekursion von Hand.** Eindimensionales CV-Modell mit Zustand $(x, v)^T$, $F = \begin{pmatrix} 1 & dt \\ 0 & 1 \end{pmatrix}$, $H = \begin{pmatrix} 1 & 0 \end{pmatrix}$. Gegeben: $\hat{x} = 2,0 \text{ m}$, $\hat{v} = 0$, $P = \text{diag}(0,04; 1,0)$ in m^2 bzw. m^2/s^2 , $dt = 0,5 \text{ s}$, $q = 0,6 \text{ m}^2/\text{s}^3$, $R = 0,25 \text{ m}^2$, Messung $z = 2,40 \text{ m}$. Bestimmen Sie P^- , die Verstärkung K (beide Einträge), $\hat{x}^+$, $\hat{v}^+$ und die Standardabweichung der Position vor und nach dem Update.
3. **Drift ist kein weißes Rauschen.** Die Odometrie entsteht aus den *gemessenen* Radgeschwindigkeiten, und auf jeder steckt σ_{wheel} . Warum darf der *Pose*-Fehler der Odometrie nicht als weißes Messrauschen mit konstantem R in den Filter? Was machen Kovarianz und NEES, wenn es doch getan wird?
4. **NEES.** Die Auswertung meldet für eine Fahrt $\overline{\text{NEES}} = 0,1$ bzw. 1 bzw. 8. Was heißt das jeweils für die angegebene Standardabweichung (Faktor, welche Richtung), und welcher der drei Werte fällt durch das Band für K3 (0,15...3,5)?

2. Theorie: prädictieren, messen, abwägen

2.1. Zustand, Prädiktion, Update, Kovarianz

Der Filter kennt einen Zustandsvektor ξ und dessen Unsicherheit, die Kovarianz P . Hier genügt zunächst $\xi = (x, y, v_x, v_y)^T$ in Weltmetern; K4 verlangt einen weiteren Freiheitsgrad für die Drehrate. Es läuft immer derselbe Zyklus: **Prädiktion** über die echte Zeitspanne dt seit dem letzten Zustand, **Update**, sobald eine Messung z mit ihrer Unsicherheit R eintrifft.

$$\xi^- = F \xi + \text{Antriebe}, \quad P^- = F P F^T + Q \quad (1)$$

$$\mathbf{y} = z - H \xi^-, \quad S = H P^- H^T + R, \quad K = P^- H^T S^{-1} \quad (2)$$

$$\xi^+ = \xi^- + K \mathbf{y}, \quad P^+ = (I - KH)P^-(I - KH)^T + K R K^T \quad (3)$$

Bei einer direkt gemessenen Größe ist $K = P^-/(P^- + R)$: ist die Prognose unsicherer als die Messung, zieht der Filter zur Messung; ist P klein, passiert wenig. Die Stellschrauben bedeuten Verschiedenes: Q sagt, was dem *Modell* fehlt (unerklärte Beschleunigung), R , was dem *Sensor* fehlt. Wer nur Q erhöht, macht den Filter unruhig; wer nur R erhöht, macht ihn träge. Beide bestimmen die Kovarianz — und damit die Kennzahlen, mit denen bewertet wird:

$$\text{NEES} = \frac{1}{N} \sum_i \frac{1}{2} \left(\frac{e_{x,i}^2}{s_{x,i}^2} + \frac{e_{y,i}^2}{s_{y,i}^2} \right), \quad \text{RMSE} = \sqrt{\langle e_x^2 + e_y^2 \rangle}, \quad \text{Verb.} = \frac{\text{RMSE}_{\text{GPS}}}{\text{RMSE}_{\text{KF}}}. \quad (4)$$

Bei zwei Freiheitsgraden ist der erwartete NEES-Wert 1; ein Einzelwert bis etwa 5,99 ist bei χ^2 -Verteilung noch unauffällig. Ohne eq. (3) in der Joseph-Form bleibt P nach Rundungsfehlern gern asymmetrisch — bei einem 4×4 -Filter von Hand vermutlich die häufigste stille Quelle für NEES weit über 1.

2.2. Das CV-Modell in Matrizen

Konstante Geschwindigkeit heißt das Modell, weil es die Beschleunigung nicht kennt. In Matrizen (Weltframe, Radreihenfolge irrelevant, Zustand (x, y, v_x, v_y)):

$$F(dt) = \begin{pmatrix} 1 & 0 & dt & 0 \\ 0 & 1 & 0 & dt \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad H = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}, \quad R = \begin{pmatrix} \sigma_{\text{gps}}^2 & 0 \\ 0 & \sigma_{\text{gps}}^2 \end{pmatrix}, \quad (5)$$

$$Q(dt) = q \begin{pmatrix} \frac{dt^4}{4} & 0 & \frac{dt^3}{2} & 0 \\ 0 & \frac{dt^4}{4} & 0 & \frac{dt^3}{2} \\ \frac{dt^3}{2} & 0 & dt^2 & 0 \\ 0 & \frac{dt^3}{2} & 0 & dt^2 \end{pmatrix}, \quad q \text{ in } \text{m}^2/\text{s}^3. \quad (6)$$

q ist die angenommene Unsicherheit der *Beschleunigung*, also dessen, was das CV-Modell nicht darstellen kann; brauchbarer Start $0,3 \dots 1,0$, empfohlen in `kf.q_acc = 0,6` (table 2). σ_{gps} wird nicht geraten, sondern aus dem Sensorprofil gelesen (`rob.config("gps.sigma_xy", 0.5)` bzw. `/sim/config`). Ohne numpy reicht handschriftliche Matrizenmultiplikation: eq. (5) ist dünn besetzt, eq. (6) blockweise — vier Zeilen Code je Achse. Zwei Dinge entscheiden über Gut oder Schlecht: dt ist die *echte* Differenz der Messstempel (nicht ein gemittelter Fixwert), und es wird bis zum *Messzeitpunkt* der eintreffenden Messung prädiziert, erst dann aktualisiert. Bei 5 Hz GPS und 0,85 m/s kosten 100 ms Rückstand 8,5 cm — das ist bereits ein Drittel des K4-Ziels.

2.3. Warum die Gyro kurzfristig und die Odometrie langfristig hilft

Zwei Fakten bestimmen die Architektur des Filters. **Erstens:** eine IMU misst Specific Force, und ihr Fehler wächst mit der Integrationsordnung. Die Werte aus table 2 sagen: Gyro-Rauschen $10^{-4} \text{ rad/s}/\sqrt{\text{Hz}}$ sind pro Stichprobe $7,1 \cdot 10^{-4} \text{ rad/s}$ — vernachlässigbar; der Gyro-Bias $5 \cdot 10^{-3} \text{ rad/s}$ ergibt in 10 s immerhin 0,05 rad (2,9°) Kursfehler, der auf 10 m Fahrt 0,25 m Querfehler kostet. Beim Accelerometer wird aus $0,05 \text{ m/s}^2$ Bias durch *zwei* Integrationen in 10 s Wegfehler von $\frac{1}{2}bt^2 = 2,5 \text{ m}$. Deshalb: Drehrate ja (einmal integrieren, klein), Beschleunigung nur kurzfristig und nie offen — die IMU ist ein Bewegungsmodell für Sekundenbruchteile, kein Ortungsgerät. **Zweitens:** die Odometrie liefert zwei verschiedene Dinge. Ihre *Pose* ist Dead Reckoning und driftet (Zufalls-spaziergang aus σ_{wheel} , kursiert durch den Drehfehler). Ihr *Twist* (v_x, v_y, ω) dagegen ist eine unmittelbare Messung: mit $r = 0,05 \text{ m}$ und $\sigma_{\text{wheel}} = 0,04$ bis $0,05 \text{ rad/s}$ steckt auf $v_x = \frac{r}{4} \sum_i w_i$ nur $\approx 1 \text{ mm/s}$ Rauschen *ohne* Drift. Der gangbarste Filter tut daher: Prädiktion mit der *gemessenen* Körpergeschwindigkeit (Odometrie-Twist, in die Welt gedreht) und der Gyro-Drehrate, Update der Position mit GPS. Wer stattdessen die Odometrie-*Pose* als Messung mit festem R einwirft, doppelt information, die schon in der Prädiktion steckt — der dritte Weg ist, die Drift als eigenen Zustandswert (Bias) mitzuführen und R mit der Zeit wachsen zu lassen.

2.4. GPS: Rauschen, Bias, Funkloch, Modellfehler

Das GPS ist der einzige absolute Sensor. Sein Rauschen ist unbekannt korreliert, also geradezu gemacht für R : $\sigma_{xy} = 0,5 \text{ m}$ in K1/K3, $0,8 \text{ m}$ in K2, $0,6 \text{ m}$ in K4 — die Roh-

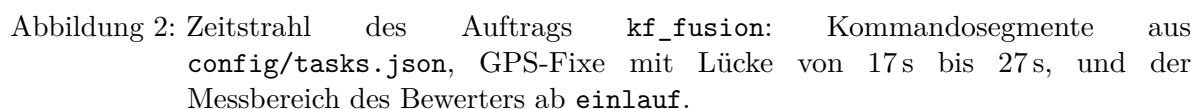
The diagram illustrates the system architecture with three main components:

- Be(robot)/truth (nur debug_truth) Simulator** (Orange box): Contains `grade.Grader`.
- Ihr Filter** (Green box): Contains `render.py`, `k: Spur, GPS, Schätzung, 2σ`, and `source/template.py, mission(rob, task)`.
- GUI** (Grey box): Contains `render.py`.

Data Flow:

- Simulator to Filter:** `<robot>/imu 100Hz`, `<robot>/gps 1bis 5Hz`, `<robot>/odom 50Hz`, `/sim/config`.
- Filter to Simulator:** `<robot>/kf/nose`, `<robot>/odom, kf/pose`, `info`.
- Simulator to GUI:** `<robot>/kf/nose` (dashed line).
- GUI to Filter:** `<robot>/kf/nose` (dashed line).

Abbildung 1: Signalfuss von Versuch 2. Der Bewerter fährt, die Simulation liefert die Sensoren, Sie liefern `kf/pose inklusive` Standardabweichung — an der gemessenen Stelle, nicht an der Wahrheit.



3. Der Simulator

3.1. Neue Themen und was sie enthalten

Versuch 1 bleibt, wie er ist; der Filter setzt darauf. table 1 zeigt den Zugang — die Pfade hängen am Roboternamen, `topic("kf", "alice")` ergibt `/alice/kf/pose`.

Tabelle 1: Neue und geänderte Themen in Versuch 2 (`types.MSG_SPECS`).

Thema	Typ (ROS 2)	Inhalt und Richtung
<code><robot>/imu</code>	<code>sensor_msgs/msg/Imu</code>	100 Hz (einstellbar), Achsen im Körperframe, <code>linear_acceleration.z</code> $\approx +g$; <code>roll/pitch</code> mitgeliefert
<code><robot>/kf/pose</code>	<code>geometry_msgs/msg/PoseWithCovarianceStamped</code>	variante <code>pose</code> inklusive 1σ (Diagonale) — die Bewertungsgröße
<code><robot>/kf/info</code>	<code>std_msgs/msg/String</code> (JSON)	Diagnose (Laufzeit, Raten, Q/R); der Auswerter ignoriert sie
<code><robot>/truth</code>	<code>geometry_msgs/msg/PoseStamped</code>	exakte Pose mit einstellbarer <code>truth.rate</code> , nur mit <code>debug_truth</code> ($= -truth$)
<code>/sim/config</code>	<code>std_msgs/msg/String</code> (JSON)	aktives Sensorprofil (<code>gps</code> , <code>odom</code> , <code>imu</code> , <code>truth</code> , <code>rate</code> , <code>seed</code>) — hier steht σ , das Sie in R einsetzen
<code><robot>/odom</code> , <code><robot>/gps</code>	wie Versuch 1	Odometrie 50 Hz mit <code>twist</code> ; GPS mit Rate und Streuung des Prüfprofils

3.2. Das IMU-Modell und seine Zahlen

`mecanum_lab/types.py` (`DEFAULT_CONFIG`) ist die Quelle aller Zahlen; `config/default.json` und `-set` darüber schreiben, das Prüfprofil des Auftrags sowieso (section 4). Die Spalte „pro Stichprobe“ ist die für den Filter relevante Größe: Dichten werden mit $\sqrt{\text{rate}/2}$ auf die Samplingrate umgerechnet, Bias-Random-Walk mit $\sqrt{1/\text{rate}}$ pro Schritt.

Warum `az` im Stand $\approx +9,81$ anzeigt. Ein MEMS-Beschleunigungsmesser misst keine Bewegung, sondern die Specific Force — die Kraft pro Masse, die ihn am freien Fall hindert. Auf ebener Fläche ist das die Normalkraft nach oben, also $+g$. Das ist kein Messfehler und kein Vorzeichenbug, sondern die Physik des Chips: wer `az` ohne Weiteres integriert, bekommt in 10 s 490 m Höhe. Für die Ebene zählen nur `ax` und `ay` — und die sind verbiasst, geneigt und überlagert, eben deshalb nur kurzfristig brauchbar. `gx/gy` sind durch dieselbe Neigung mitbedingt, `gz` ist das Hauptsignal in 2D.

Nachmessen statt glauben. Ein Standlauf über 5 s (Seed 1, Roboter *alice*, 502 Stichproben) liefert $az = 9,74 \pm 0,014 \text{ m/s}^2$ — die Streuung ist exakt die gerechnete Dichte, der Unterschied zu 9,81 ist der `accel_bias` der z -Achse — und ein Gyro-Mittel von $-5,4 \cdot 10^{-3} \text{ rad/s}$. Genau dieses Mittel ist Ihre Kalibrierung: IMU im Stillstand mitteln (vor der Fahrt oder in jederr Pause), Mittel abziehen — so steht die Kalibrierung später auf dem echten Roboter.

Tabelle 2: Sensorik-Einstellungen aus DEFAULT_CONFIG (Versuch-2-Auszug).

Größe	Wert	Bedeutung für Ihren Filter
<i>IMU (imu) — MEMS-Klasse, absichtlich so kaputt</i>		
rate	100 Hz	K4 fährt 200 Hz; <code>-set imu.rate=400</code> ist erlaubt
accel_noise	$2 \text{ e-}3 \text{ m/s}^2/\sqrt{\text{Hz}}$	$\Rightarrow 14 \text{ mm/s}^2$ pro Stichprobe
accel_bias	$0,05 \text{ m/s}^2$	je Roboter und Achse neu gezogen (Seed); doppelte Integration: 2,5 m in 10 s
accel_bias_walk	$5 \text{ e-}4 \text{ m/s}^2/\sqrt{\text{s}}$	der Bias bleibt nicht, wo er war: nach 10 s streut er mit $1,6 \text{ mm/s}^2$
accel_scale	$1 \text{ e-}3$ (0,1 %)	verschwindet nicht durch Mitteln
gyro_noise	$1 \text{ e-}4 \text{ rad/s}/\sqrt{\text{Hz}}$	$\Rightarrow 0,7 \text{ mrad/s}$ pro Stichprobe
gyro_bias	$5 \text{ e-}3 \text{ rad/s}$	0,05 rad Kursfehler in 10 s — mitteln hilft
gyro_bias_walk	$2 \text{ e-}5 \text{ rad/s}/\sqrt{\text{s}}$	nach 10 s streut der Bias selbst mit $0,06 \text{ mrad/s}$ — klein, aber er verschwindet nicht von allein
tilt_sigma, tilt_tau	6 mrad, 0,4 s	Federung wackelt, Schwerkraft sickert in ax/ay : $g\varphi \approx 0,06 \text{ m/s}^2$
vibration, vibration_hz	$0,08 \text{ m/s}^2$, 16 Hz	Fahrwerksschwingung, mittelbar (Tiefpass oder Mittelung)
startup, startup_bias	0,4 s, $0,5 \text{ m/s}^2$	Einschwingen des Treibers — in dieser Zeit nicht kalibrieren
gravity	$9,81 \text{ m/s}^2$	steckt voll in az
<i>Übrige Sensorik</i>		
gps.rate, gps.sigma_xy	5 Hz, 0,06 m	Versuch 1; die KF-Aufträge setzen 1 bis 5 Hz und 0,5 bis 0,8 m
gps.bias_xy, gap, bias_step	[0, 0], null, null	Versatz, Funkloch $[t_0, \text{dauer}]$, springender Bias
odom.rate, sigma_wheel	50 Hz, $0,04 \text{ rad/s}$	auf jeder Radmessung — daraus wird die Drift
truth.rate	20 Hz	Rate der exakten Pose (Bewertung)
kf.rate, kf.q_acc, kf.q_turn	20 Hz, $0,6 \text{ m}^2/\text{s}^3$, 0,02	<i>Empfehlung an Sie</i> — die Simulation benutzt diesen Block nicht

3.3. Auswertung in der GUI

Die Anzeige ist dieselbe wie in Versuch 1, neu ist die Überlagerung der Schätzung. Die Hilfeliste im HUD: **q** Ende | **SPACE** Pause | **l** Scan | **t** Spur | **k** Schätzung | **0** alles | **1..9** Roboter | **+/-** Zoom. Die Taste **k** schaltet GPS-Kreuze, die Schätzung und die 2σ -Ellipse drumherum — die Ellipse ist Ihr NEES mit Augen: sie muss die Wahrheit *umfahren*, nicht umschließen-wollen. Im HUD steht darüber der aktuelle Abstand Ihrer Schätzung zur Wahrheit (**kf_err**) — nur sichtbar, nicht im Filter benutzt. Für den Bericht gehört die Messreihe in eine Datei: `-log messung.csv` schreibt alle 0,05 s Simulationszeit eine Semikolon-Zeile mit **t**, **x_wahr**, **x_gps**, **t_alt_gps**, **x_kf**, **sx_kf**, **ax_imu** und mehr (Spaltenkopf aus `mecanum_lab/logbook.py`); `python3 tools/kfplot.py messung.csv` malt Zeitreihen, RMSE und — ohne matplotlib — ein ASCII-Diagramm.

4. Inbetriebnahme

4.1. Installation

```
./install.sh          # einmalig: prueft pygame, ROS 2 und die Tests
# alternativ:  sudo apt install python3-pygame      (reicht fuer den Stub-
Modus)
./lab docs            # Themen, Auftraege und Beispiele auf einen Blick
```

Getestet ist Ubuntu 24.04 mit ROS 2 Kilted/Jazzy; ohne ROS läuft alles über den In-Prozess-Bus (MECANUM_ROS=stub) — dann zeigt `ros2 topic list` natürlich nichts.

4.2. Werkstatt: den eigenen Filter fahren

```
# 1) Werkstatt mit Fenster: die Sim faehrt, du schaetzt, truth ist an
./lab run --world arena --task kf_gps --robot alice --robots alice \\\
--controller student/kf_template.py --truth --log messung.csv

# 2) Bewerten wie der Abgabepuefer (ein Auftrag, oder alle vier)
./lab grade --task kf_gps --controller student/kf_template.py --log messung.
csv
./lab grade --task kf_alle --controller student/kf_template.py \\\
--log messung.csv --json bericht.json
python3 tools/kfplot.py messung.csv

# 3) Sensorik frei verstellen, keine JSON-Datei noetig
./lab grade --task kf_fusion --controller student/kf_template.py \\\
--set gps.sigma_xy=1.2 --set imu.rate=400 --set gps.gap='[14,8]'
```

Drei Fallen beim ersten Start. *Erstens:* `./lab run` spawn only Roboter aus `-robots` — ohne `-robots` `alice` läuft eine leere Halle, und das Protokoll bleibt leer. *Zweitens:* ohne `-truth` (bzw. `debug_truth`) gibt es kein `truth`-Thema, der Bewerter kann dann nicht messen. *Drittens:* der eigene Knoten darf `wheel_speeds` und `cmd_vel` nicht anfassen (section 1). Unter ROS gilt außerdem:

```
ros2 topic hz /alice/imu          # ~100 Hz
ros2 topic echo /alice/imu --once  # az im Stand: rund +9,81
ros2 topic echo /sim/config --once # sigma_xy, gap, imu-rate des
laufenden Profils
ros2 topic hz /alice/kf/pose       # Ihre Rate -- K4 verlangt
mindestens 10 Hz
```

4.3. ROS-2-Betrieb mit dem Launch-File

Ohne Launch-File starten Sie die Simulation mit `./lab sim`; mit ihm kommt alles aus einer Zeile — und *jedes* Argument ist eine `-set`-Angabe an die Simulator-Konfiguration (table 3). So verändern Sie Sensorwerte für den Bericht systematisch, ohne JSON-Dateien anzufassen.

```

source /opt/ros/kilted/setup.bash          # falls ROS nicht automatisch
gefunden wird
ros2 launch launch/kf.launch.py --show-args      # alle Argumente
erklären
ros2 launch launch/kf.launch.py headless:=true aufgabe:=kf_fusion \\\
controller:=student/kf_template.py protokoll:=messung.csv
ros2 launch launch/kf.launch.py bewerten:=kf_alle \\\
controller:=student/kf_solution.py "gps_luecke:=[10,14]" imu_rate:=200
ros2 launch launch/kf.launch.py rviz:=true      # Ansicht mit
Kovarianzellipse

```

Tabelle 3: Die Argumente, die im Versuch gestellt werden (Auswahl; vollständige Liste: `ros2 launch launch/kf.launch.py -show-args`).

Argument	Wirkung	Standard
aufgabe	kf_gps kf_fusion kf_kovarianz kf_dynamik	kf_fusion
controller	Ihr Filterknoten, relativ zum Quellbaum	student/kf_template.py
bewerten	Auftrag oder Gruppe (z. B. kf_alle); leer = nicht bewerten	—
protokoll	CSV-Messprotokoll für tools/kfplot.py	—
gps_rate, gps_sigma	Messungen pro Sekunde, Streuung je Achse in m	Profil
gps_luecke	Funkloch als [start, dauer] in s nach Auftragsbeginn	keins
gps_bias_step	springender Bias [start, dauer, dx, dy] in m	keiner
imu_rate	IMU-Stichproben pro Sekunde	100 (K4: 200)
imu_accel_bias, imu_accel_walk	Bias 1σ in m/s^2 , Bias-Random-Walk in $\text{m/s}^2/\sqrt{s}$	$0,05 \cdot 5\text{e-}4$
imu_neigung, imu_vibration	Neigung 1σ in rad (Schwerkraft sickert in a_x), Schwingung in m/s^2	$0,006 \cdot 0,08$
wahrheit, seed, sekunden	truth-Thema, Rausch-Seed, Abbruch nach s	true · 1 · 0

Ein Detail mit Folgen: `gps_luecke` und `gps_bias_step` stehen in Sekunden *nach Auftragsbeginn*, nicht nach Programmstart. Der Bewerter kann das Funkloch also jederzeit auslösen, und Ihre Messung bleibt dieselbe.

4.4. Drei Regeln, die den Lauf retten

Erstens: Messstempel statt Wanduhr. Jede Messung trägt t — die Simulationszeit, in der sie entstanden ist. Benutzen Sie diese Stempel für dt , für Q und dafür, ob eine Messung schon verarbeitet ist. Wer stattdessen `time.time()` aufruft, bekommt in der Bewertung einen falschen dt : das Beschleunigungsskript `tools/fastgrade.py` rechnet 25 Simulationssekunden in einer Sekunde Wanduhr, und Ihr Filter hält dann eine 5-Sekunden-Lücke für 200 Millisekunden. *Zweitens: Sie schätzen, Sie fahren nicht.* In den

KF-Aufträgen setzt der Bewerter `cmd_vel`, die Simulation fährt die Kommandofolge aus `config/tasks.json` blind. Ihr Knoten liest Sensoren und antwortet auf `kf/pose` — `send_wheels()`, `publish_cmd_vel()` und `rob.truth()` sind in diesem Versuch tabu (der Bewerter merkt das an der fehlenden Fahrt und an der Kontaktzahl), sonst urteilt der Bewerter über eine Fahrt, die Sie nie gefahren sind.

Drittens: ein neuer Auftrag beginnt wirklich von vorn. Wenn sich `rob.task()` ändert, wird der Roboter an der Spawn-Pose abgesetzt und das Sensorprofil gewechselt. Beenden Sie dann Ihren Mess-Loop, lesen Sie `rob.sensor_profil()` erneut vor und — das ist der Fall, der am meisten Punkte kostet — werfen Sie die *letzte* Messung des vorigen Auftrags: Der Bus merkt sich die letzte Meldung, und ein GPS-Fix von vor dem Respawn ist für den neuen Auftrag eine Lüge. Prüfen Sie also `fix.t` gegen den Beginn Ihrer Mission (die Musterlösung führt dafür `KF.start`). Die Gyro-Bias-Kalibrierung dürfen Sie ruhig wiederholen — der Zustand und P gehören dagegen nicht auf null zurückgesetzt, sonst werfen Sie Information weg, die Sie gerade erst gesammelt haben.

5. Aufgaben K1 bis K4

Alle vier Aufträge fahren dieselbe Logik: Der Bewerter setzt Ihren Roboter an die Spawn-Pose, fährt die in `config/tasks.json` hinterlegte Kommandofolge, zeichnet `truth` und `kf/pose` auf und vergleicht. Gemessen wird erst nach der Einlaufzeit (4s), damit Ihre Bias-Kalibrierung im Stand zählt. table 4 listet die Prüfgrenzen; `./lab grade -task kf_alle -controller student/kf_template.py` zeigt Ihnen vorab dieselben Zahlen, die in der Abgabe zählen.

Tabelle 4: Prüfgrenzen der vier Teilaufträge (Quelle: `config/tasks.json`).

Auftrag (Punkte)	Sensorprofil	Prüfgrenzen
K1 <i>kf_gps</i> (30)	GPS 5 Hz, $\sigma = 0,5$ m	RMSE $\leq 0,42$ m, Verbesserung $\geq 1,6$, Maxfehler $\leq 1,25$ m, Rate ≥ 5 Hz
K2 <i>kf_fusion</i> (30)	GPS 1 Hz, $\sigma = 0,8$ m, Funkloch 15...23s; Odometrie, IMU	RMSE $\leq 0,80$ m, Verbesserung $\geq 2,5$, Fehler im Funkloch $\leq 1,8$ m
K3 <i>kf_kovarianz</i> (20)	wie K1	mittleres NEES in $[0,15 \dots 3,5]$, RMSE $\leq 0,42$ m
K4 <i>kf_dynamik</i> (10, Bonus)	GPS 5 Hz, $\sigma = 0,6$ m, IMU 200 Hz, 0,85 m/s mit Schwingung	RMSE $\leq 0,35$ m, Verbesserung $\geq 1,8$, Maxfehler $\leq 0,9$ m, Rate ≥ 10 Hz

K1 — Positionsschätzung aus GPS allein. Bauen Sie das CV-Modell $\xi = (x, y, v_x, v_y)^T$ mit $F(dt)$ und dem über dt integrierten Beschleunigungsrauschen Q (eqs. (5) and (6)) und aktualisieren Sie mit der GPS-Position *zum Stempel* `fix.t`. Ziel ist das Verständnis der Rekursion, nicht der Trickkiste: ein einzelner, ehrlicher KF reicht.

K2 — GPS, Odometrie und IMU verschmelzen. Bei 1 Hz GPS mit 0,8 m Streuung ist der *absolute* Fehler rauschbegrenzt — bewertet wird deshalb die Verbesserung gegen das Rohsensorignal. Jetzt hilft der Odometrie-*twist* als Eingangsgröße der Prädiktion (in die Welt gedreht) und der Gyro für die Drehrate — beides mit Bias, der erst im Stand gemessen werden will. Im Funkloch bleibt nur das Bewegungsmodell: Hier entscheidet sich, ob Sie Q physikalisch begründen oder raten.

K3 — Wissen, wie gut man schätzt. Dieselbe Fahrt wie K1, aber bewertet wird die *angegebene* Unsicherheit: `kf/pose` trägt σ_x, σ_y aus P . Vergrößern Sie Q , wenn das NEES über 3 läuft; verkleinern Sie es, wenn es unter 0,3 fällt — und schreiben Sie den Zielkonflikt in den Bericht.

K4 — Schnelle Fahrt, treue Schätzung. 0,85 m/s mit einer Sinusüberlagerung in der Drehrate, IMU mit 200 Hz. Hier zahlt sich aus, dass Ihr Filter mit der *Messrate* läuft statt mit einer Rateschleife.

Typische Fehlerbilder, an denen Sie Ihre Implementierung erkennen: Die Schätzung läuft der Messung konstant hinterher (F ohne dt , $Q = 0$); die Kovarianz schrumpft und steigt nie wieder (Joseph-Form oder P -Aktualisierung vergessen); der Roboter wird über den Gyro gedreht, obwohl der Kurs in der Welt gebraucht wird (Vorzeichen/Drehung der Körpergeschwindigkeit); das Funkloch wird mit einem R von 10^{-6} überbrückt (dann folgt die Schätzung der Odometrie blind und table 4 K2 fällt durch).

6. Auswertung und Bericht

Der Bericht gehört Ihnen — aber die Auswertung liefert Zahlen, die Sie erklären müssen. Er erwartet:

- **Bahn und Fehlerbild.** Ein Plot aus `tools/kfplot.py`: Wahrheit, rohes GPS, Odometrie, Ihre Schätzung — darunter der Fehler gegen die Wahrheit über der Zeit. Beschreiben Sie, *wo* der Fehler wächst (Kurven? Funkloch? nach dem Aufgabenwechsel?).
- **Eine Q/R -Tabelle.** Welche Werte haben Sie benutzt, wie sind Sie darauf gekommen (Dichte $\rightarrow Q$, `gps.sigma_xy` $\rightarrow R$), und was passiert bei Faktor 3 in beide Richtungen? Vier Messpunkte genügen — aber es müssen Messpunkte sein, keine Vermutungen.
- **NEES-Diskussion zu K3.** Ihr Mittelwert, die Streuung innerhalb der Fahrt, und was Sie geändert haben, um ins Band zu kommen.
- **Gemessene IMU-Bias-Drift.** Mitteln Sie `imu` im Stand über 10 s vor und nach der Fahrt, geben Sie die Differenz pro Achse an und vergleichen Sie mit table 2. Daraus folgt Ihre Begründung, warum die IMU nicht offen integriert wird.
- **Nutzen der Odometrie im Funkloch.** RMSE im Loch mit und ohne Odometrie-Prädiktion (zwei Protokolle, ein Satz Fazit). Wenn Sie behaupten, das Modell reiche auch ohne — messen Sie es.
- **Ein Satz zur Modellwahl.** K4 fährt schneller und schwingt: Welche Zustandsgröße hätten Sie noch gebraucht (Drehrate im Zustand? Bias als Zustand?), und was hätte es gekostet?

Rechnen Sie mit dem Protokoll, nicht mit dem Fenster: `-log messung.csv` schreibt truth, gps, odom, kf, imu und die Geschwindigkeiten in eine CSV-Datei, die auch ohne matplotlib auswertbar ist (`-list` zeigt die Spalten).

A. Formelsammlung

CV-Modell (geradlinig, Weltframe):

$$F(dt) = \begin{pmatrix} 1 & 0 & dt & 0 \\ 0 & 1 & 0 & dt \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad Q(dt) = q \begin{pmatrix} \frac{dt^3}{3} & 0 & \frac{dt^2}{2} & 0 \\ 0 & \frac{dt^3}{3} & 0 & \frac{dt^2}{2} \\ \frac{dt^2}{2} & 0 & dt & 0 \\ 0 & \frac{dt^2}{2} & 0 & dt \end{pmatrix}$$

mit der Beschleunigungs-Dichte q in m^2/s^3 — der Simulator startet mit $q = 0,6$ (`kf.q_acc`), die Drehrate mit $q_{\text{gier}} = 0,02$.

Rekursion: $\xi^- = F\xi$, $P^- = FPF^\top + Q$, $y = z - H\xi^-$, $S = HP^-H^\top + R$, $K = P^-H^\top S^{-1}$, $\xi = \xi^- + Ky$, $P = (I - KH)P^-$. Für die Positions-Messung ist H die Auswahlmatrix mit den beiden Einsen in der x - und y -Spalte, $R = \text{diag}(\sigma_{\text{gps}}^2, \sigma_{\text{gps}}^2)$ mit σ_{gps} aus `gps.sigma_xy`.

Umrechnungen, die man nachschlagen darf: Weiße Dichte d [Einheit/ $\sqrt{\text{Hz}}$] $\rightarrow$ Streuung pro Stichprobe $d\sqrt{\text{rate}/2}$. Bias-Random-Walk $w \rightarrow$ Streuung pro Schritt $w/\sqrt{\text{rate}}$, nach T Sekunden $w\sqrt{T}$. Fester Bias b über T Sekunden doppelt integriert: Wegfehler $\frac{1}{2}bT^2$.

Konsistenz: $\overline{\text{NEES}} = \frac{1}{N} \sum_i [(e_{x,i}^2/\sigma_{x,i}^2 + e_{y,i}^2/\sigma_{y,i}^2)/2]$; erwarteter Wert 1, ein Einzelwert bis 5,99 ist bei zwei Freiheitsgraden noch unauffällig (χ^2 , 95 %). Angegeben werden $\sigma_x = \sqrt{P_{11}}$, $\sigma_y = \sqrt{P_{22}}$ — nicht die Wurzel der Spur.

B. Lösungen der Vorfragen

1. **IMU.** Eine Stichprobe: $\sigma_a = 0,002 \cdot \sqrt{100/2} = 0,014 \text{ m/s}^2$ (14 mm/s²). Der Faktor $\sqrt{\text{rate}/2}$ kommt daher, dass die Dichte über die äquivalente Rauschbandbreite integriert wird — bei Abtastrate f_s ist das $f_s/2$. Fester Bias $b = 0,05 \text{ m/s}^2$: nach $T = 10 \text{ s}$ ist $\Delta v = bT = 0,5 \text{ m/s}$ und der Wegfehler $\frac{1}{2}bT^2 = 2,5 \text{ m}$. Das weiße Rauschen trägt nach derselben Zeit nur $\sigma_a T^2 N^{3/2}/\sqrt{3} \approx 0,03 \text{ m}$ bei ($N = 1000$ Stiche) — der Bias dominiert um zwei Größenordnungen. *Folge:* Die IMU taugt als *Bewegungsmodell* mit kurzem Gedächtnis, nicht als Positionsmesser; der Bias muss geschätzt (oder zumindest durch Mittelung im Stand klein gemacht) werden.

2. **Rekursion.** Mit $dt = 0,5$ und $q = 0,6$:

$$FPF^\top = \begin{pmatrix} 0,04 + dt^2 \cdot 1,0 & dt \cdot 1,0 \\ dt \cdot 1,0 & 1,0 \end{pmatrix} = \begin{pmatrix} 0,29 & 0,50 \\ 0,50 & 1,00 \end{pmatrix}, \quad Q = 0,6 \begin{pmatrix} 0,0417 & 0,125 \\ 0,125 & 0,5 \end{pmatrix},$$

also $P^- = \begin{pmatrix} 0,315 & 0,575 \\ 0,575 & 1,30 \end{pmatrix}$ und $S = 0,315 + 0,25 = 0,565$. Verstärkung $K = P^-H^\top/S = (0,315/0,565, 0,575/0,565) = (0,56, 1,02)$. Innovation $y = 2,40 - 2,00 =$

0,40, damit $\hat{x}^+ = 2,00 + 0,56 \cdot 0,40 = 2,22$ m und $\hat{v}^+ = 0 + 1,02 \cdot 0,40 = 0,41$ m/s — die Verstärkung im zweiten Eintrag darf größer als 1 sein, sie wirkt auf eine Geschwindigkeit. Position: $\sigma^- = \sqrt{0,315} = 0,56$ m vor, $\sigma^+ = \sqrt{0,315 - 0,56^2 \cdot 0,565} = 0,37$ m nach dem Update.

3. **Odometrie-Drift.** Der Posenfehler der Odometrie ist die Summe aller bisher begangenen Fehler: er ist zeitlich korreliert, wächst mit der zurückgelegten Strecke und hat einen Vorzeichen-zusammenhang (ein zu großer Radradius bleibt zu groß). Weißes Rauschen mit festem R beschreibt aber einen Fehler, der mit jeder Messung vergessen wird. Folge: Der Filter hält jede neue Odometrie-Pose für eine unabhängige Messung, P schrumpft gegen null, die Kovarianz ist zu klein — das NEES steigt weit über 1, und im GPS-Funkloch driftet die Schätzung unhinterfragt mit. Richtig: Odometrie als *Eingang* (Geschwindigkeit) in die Prädiktion benutzen und den Rest als Prozessrausch Q (wachsend mit dt) oder als zusätzlichen Bias-Zustand modellieren.
4. **NEES.** 0,1: Die angegebene Standardabweichung ist im Mittel $\sqrt{1/0,1} \approx 3,2$ -mal zu groß — der Filter ist unnötig vorsichtig und traut dem Messwert zu wenig. 1: konsistent, die Streuung der Fehler passt zur Angabe. 8: zu klein um $\sqrt{8} \approx 2,8$ -mal — übermütig, und damit gefährlich, weil die Anwendung (Regelung, Sicherheit) sich auf die Angabe verlässt. Durch das Band $[0,15; 3,5]$ für K3 fallen 0,1 und 8, nur 1 besteht. (Der untere Rand ist absichtlich locker — 0,15 entsprechen einer 2,6-mal zu großen Angabe: lässig, aber nicht gefährlich. Über 3,5 wird es eng, weil selbstsichere Falschangaben im Labor niemandem helfen.)

C. Themen- und API-Übersicht

Der Zugang zu Simulation und Themen läuft über `mecanum_lab/robot_io.py`; unter ROS 2 wie im Stub-Lauf gleich. Für Versuch 2 sind diese Methoden zuständig:

Dazu die Themen aus table 1 und, für Neugierige, der Blick von außen: `ros2 topic echo /alice/imu -once, ros2 topic hz /alice/kf/pose, ros2 topic echo /sim/config -once`. Wer ohne ROS arbeitet, nutzt dieselben Themen über den In-Prozess-Bus — nur `ros2` gibt es dort nicht.

Tabelle 5: Studentische API in Versuch 2 (Auszug aus `robot_io.py`).

Aufruf	Inhalt	Brauchen Sie für
<code>rob.gps()</code>	<code>Gps(t, x, y, theta)</code> — langsam, verstreut, im Funkloch <code>None</code>	K1–K4
<code>rob.odom()</code>	<code>Odom(t, x, y, theta, vx, vy, omega)</code> — 50 Hz, driftend	K2, K4
<code>rob.imu()</code>	<code>Imu(t, ax, ay, az, gx, gy, gz, roll, pitch)</code> im Körperframe	K2, K4
<code>rob.scan()</code>	Laser-Entfernungen (Versuch 1, hier nicht bewertet)	—
<code>rob.age("gps")</code>	Alter der letzten Messung in s — <code>1e9</code> : nie eine gehabt	überall
<code>rob.sensor_profil()</code>	aktives Prüfprofil als dict — hier steht σ für R	überall
<code>rob.config("gps.sigma_xy", 0.5)</code>	einzelner Konfigurationswert, mit Standardwert	R, Q
<code>rob.task()</code>	Name des laufenden Auftrags — ändert sich je Teilaufgabe	Loop-Ende
<code>rob.running()</code>	läuft die Simulation noch?	Loop-Bedingung
<code>rob.spin(dt)</code>	Messungen verarbeiten, ohne die Wanduhr zu befragen	Loop-Körper
<code>rob.send_kf(x, y, theta, sx, sy, sth)</code>	Ihre Schätzung samt 1σ — die Bewertungsgröße	K1–K4
<code>rob.kf()</code>	was zuletzt auf <code>kf/posestand</code> (Kontrolle des eigenen Knotens)	Debug
<code>rob.truth()</code>	exakte Pose — <i>im Filterpfad tabu</i> , für Tests erlaubt	Auswertung